AIORI-3 HACKATHON

Minimal Two-Node Cloud Hypervisor VM Controller



A unified control plane for managing and monitoring virtual machines across two hypervisor nodes.

PROBLEM STATEMENT • PS-023

TEAM DEBUG CREW

Sneha Singh · Tannu Priya

Sharda University

Problem Statement

Managing VMs across multiple hypervisor nodes can require separate tools and manual coordination.

01 **Limited visibility**

Node health is not always visible in one place.

02 **Resource comparison**

CPU and memory availability can be difficult to compare.

03 **Manual placement**

Operators may need to decide manually where a VM should run.

04 **Operation status**

Results and failures can be hard to track consistently.

GOAL **A lightweight controller with a unified view of two hypervisor nodes and simpler VM management.**

Proposed Solution

A centralized dashboard and controller API connect operators to two hypervisor nodes through one interface.

01

Monitor

View health for both nodes.

02

Inventory

See VMs and resource availability.

03

Recommend

Suggest a node based on available capacity.

04

Validate

Check requests before execution.

05

Track

Show operation status and event history.

ONE CONTROL PLANE

Two nodes • Validated operations • Clear status

Objectives & Scope

OBJECTIVES

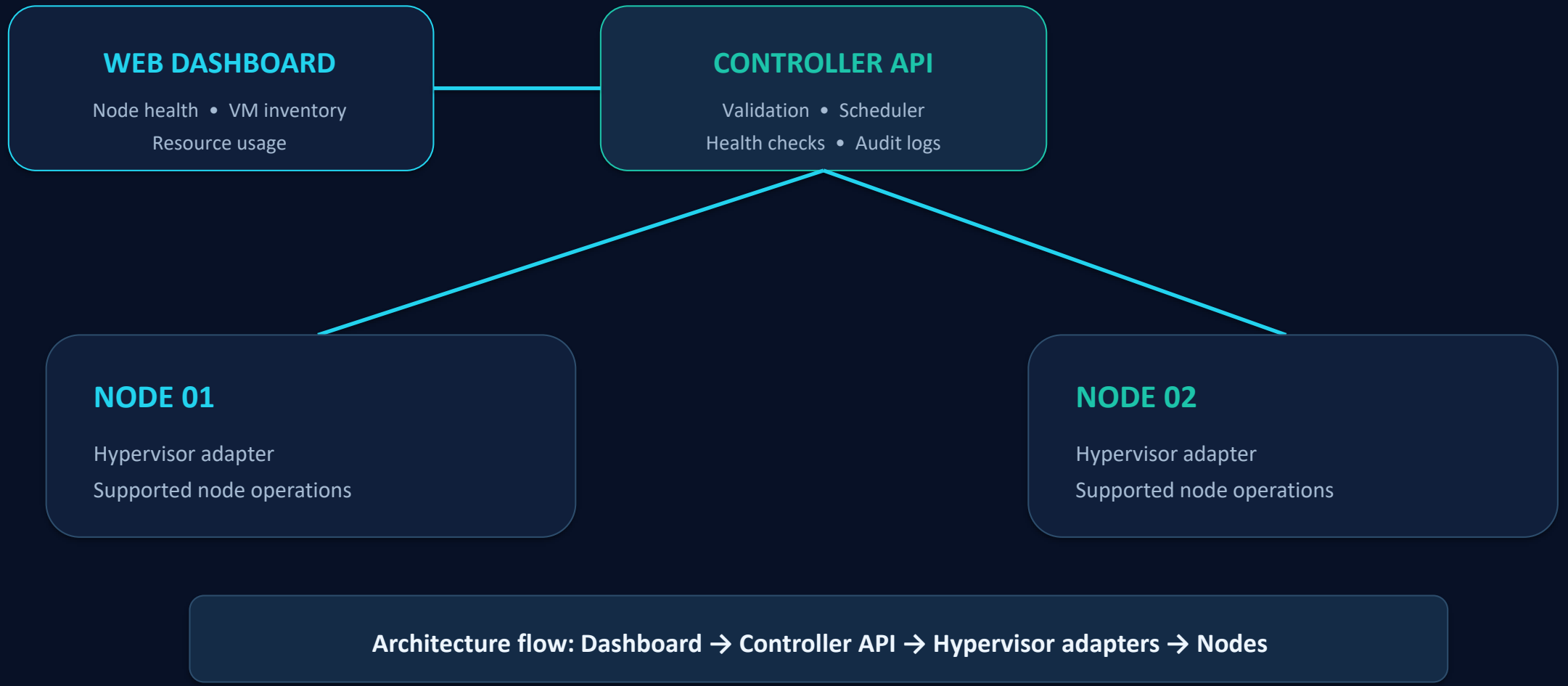
- Display the status of both hypervisor nodes.
- Summarize VMs and available resources.
- Support a controlled VM management workflow.
- Recommend placement based on resource availability.
- Record operation results for debugging and transparency.

INITIAL SCOPE

- Two hypervisor nodes
- Hypervisor APIs available in the development environment
- Core monitoring, validation and supported VM actions

Not claimed unless implemented and tested: live migration, automatic high availability, or advanced orchestration.

System Architecture



Proposed Technology Stack

COMPONENT	PROPOSED TECHNOLOGY	PURPOSE
Frontend	React + TypeScript	Interactive dashboard
Styling	Tailwind CSS	Responsive interface
Backend	Node.js + Express	API and controller logic
Hypervisor integration	Supported hypervisor API (e.g., KVM/libvirt where available)	VM and node operations
Data storage	Lightweight database, if required	Configuration and event history
Testing	API tests + simulated node responses	Verify workflows and failures

Technology choices are proposed; final selections must match the available development environment.

VM Management Workflow

01

OBSERVE

Read node health, VM inventory and resources.

02

SELECT

Choose a VM action and eligible node.

03

VALIDATE

Check permissions, inputs and available capacity.

04

EXECUTE

Send request through the supported hypervisor adapter.

05

VERIFY

Refresh state and record the operation result.

EXPECTED OUTCOME

Every request displays a clear status—including failures and uncertain results.

Features & Security

MAIN FEATURES

- Centralized node monitoring.
- VM inventory and resource summaries.
- Resource-aware placement recommendations.
- VM operation status and history.
- Error reporting and event logging.

SECURITY MEASURES

- Keep credentials and secrets out of GitHub.
- Validate all user inputs.
- Allow only explicitly supported VM operations.
- Use role-based access controls where implemented.
- Restrict API access and log administrative actions.

SAFETY PRINCIPLE • Verify the result before reporting an operation as successful.

Implementation Roadmap

MILESTONE 1	**Dashboard**	Build the interface and show representative data for both nodes.
MILESTONE 2	**Controller API**	Implement endpoints, input validation and operation-status handling.
MILESTONE 3	**Hypervisor integration**	Connect to the supported API, or clearly label a mock adapter.
MILESTONE 4	**Testing & demo**	Test successful operations, invalid requests and unavailable-node scenarios.

Build incrementally: demonstrate what is implemented, and label simulated functionality clearly.

Testing & Evaluation

TESTING SCENARIOS

- Both nodes are reachable.
- One node becomes unavailable.
- Resource information is missing or outdated.
- A VM operation succeeds or fails.
- An API request contains invalid input.
- The dashboard refreshes after an operation.

EVALUATION QUESTIONS

- Is node and VM status accurate?
- Are invalid or disallowed operations rejected?
- Does the displayed result match observed state?
- Are important events recorded?
- Can the team demonstrate the workflow reliably?

Report actual results only after running tests; do not present proposed targets as achieved results.

Team, Expected Impact & Demo

TEAM DEBUG CREW

Sneha Singh

Tannu Priya

Responsibilities: add after confirming actual work division.

EXPECTED IMPACT

- Easier monitoring
- Clearer placement decisions
- Consistent supported operations
- Better error visibility

CLOSING STATEMENT

“Our goal is to make two-node virtual machine management simpler, more transparent and easier to monitor through a lightweight centralized controller.”

DEMO WALKTHROUGH

01 Introduce PS-023 and the problem

02 Show both nodes on the dashboard

03 Display VM inventory and resources

04 Run one validated VM workflow

05 Show result and event history

06 Explain limitations and next steps

Before submission: update the deck to match what the team has actually implemented and tested.